

AGENT ORCHESTRATION SYSTEM

Production-Grade Multi-Agent Platform
Step-by-Step Build Guide · 2026 Edition

AI ENGINEER	FAANG-READY	PRODUCTION-GRADE	2026
-------------	-------------	------------------	------

Project	Project 15 – Agent Orchestration System
Target Role	AI Engineer (FAANG / Top AI Labs)
Stack	FastAPI · LangGraph · Redis · ChromaDB · PostgreSQL
Infra	Docker · Kubernetes · GCP/AWS · OpenTelemetry
Estimated Build	6–8 weeks (solo) · 3–4 weeks (team)



TABLE OF CONTENTS

1.	System Architecture & Design Philosophy	3
2.	Technology Stack & Justification	5
3.	Phase 1 – Foundation & Local Dev Setup	6
4.	Phase 2 – Core Orchestration Engine	8
5.	Phase 3 – Memory Systems	11
6.	Phase 4 – Tool Registry & Security	13
7.	Phase 5 – Human-in-the-Loop (HITL)	15
8.	Phase 6 – Observability & Tracing	16
9.	Phase 7 – Productionisation & Kubernetes	18
10.	Phase 8 – Evaluation & Testing	20
11.	Trade-offs & Decision Matrix	21
12.	Alternatives Comparison	23
13.	AI-Assisted Development Workflow	25
14.	Portfolio Presentation Strategy	26

1

SYSTEM ARCHITECTURE & DESIGN PHILOSOPHY

2026 Production Architecture

The architecture below is enhanced beyond the original Project 15 spec with four key 2026 improvements: **streaming-first design**, **model routing layer**, **sandboxed tool execution**, and **multi-tenant memory isolation**. Every component is chosen for minimal future migration cost.

CLIENT / API LAYER	FastAPI (POST /tasks · GET /tasks/{id}) · SSE streaming endpoint · WebSocket for live progress
ORCHESTRATION ENGINE	LangGraph state machine · Supervisor → Planner → Executor → Reviewer cycle · conditional edges
MODEL ROUTER	Route by complexity: Claude Sonnet (reasoning) · Claude Haiku / GPT-4o-mini (formatting, cheap tasks)
SPECIALIST AGENTS	Research Agent · Data Agent · Code Agent · Writing Agent · each owns a domain + toolset
TOOL REGISTRY	Custom registry + MCP schema · sandboxed exec (gVisor/Firecracker) · rate limiter · call logger
SHORT-TERM MEMORY	Redis · task context · plan state · partial results · TTL-based auto-expiry per task
LONG-TERM MEMORY	Qdrant (vectors) + PostgreSQL · importance scoring · episodic traces · tenant-partitioned
HITL QUEUE	PostgreSQL + Celery · SLA deadlines · auto-escalation · React approval UI
OBSERVABILITY	OpenTelemetry → Grafana · LangSmith (dev) · cost dashboard · replay system
INFRA	Docker Compose (dev) → Kubernetes + Helm (prod) · GCP Cloud Run / AWS ECS option

Data Flow – End-to-End Example

Task: "Research Q3 sales trends, build a chart, write a summary"

1	Client POST /tasks Returns task_id immediately. SSE endpoint opened for streaming updates.
2	Supervisor receives request Queries long-term memory for similar past tasks via semantic search.
3	Planner Node → Execution Plan Subtask A: Research (web_search) → Subtask B: Data (extract_numbers, depends A) → Subtask C: Code (python_chart, depends B) → Subtask D: Writing (depends C) Plan stored in Redis: HSET task:{id}:plan ...
4	Executor routes to Specialists Parallel execution where dependencies allow. Model router selects cheapest capable model per subtask.
5	Each Specialist calls Tools Tool Registry validates, rate-limits, and logs every call. Code tools run in gVisor sandbox.
6	Reviewer Node validates LLM-as-judge scores each output. Confidence < 0.7 → HITL escalation.

- | | |
|---|--|
| 7 | Human Review (if needed)
Celery task created, SLA = 10 min. Reviewer sees context + can approve/edit/reject. |
| 8 | Supervisor synthesises
Merges all subtask results. Streams final_answer event via SSE. |
| 9 | Trace persistence
Full OpenTelemetry trace saved. Long-term memory updated with task embeddings. |

2

TECHNOLOGY STACK & JUSTIFICATION

Layer	Chosen Tech	Why (2026)	Migration Risk
API	FastAPI 0.115+	Async-native, SSE support, auto OpenAPI docs, battle-tested at scale	Low - standard HTTP
Orchestration	LangGraph 0.4+	State machine with cycles, conditional edges, built-in checkpoints	Medium - Anthropic-backed
LLM Provider	Anthropic Claude (primary) + OpenAI GPT-4o (fallback)	Claude for reasoning; GPT-4o for cheap steps	Low - proxy via LiteLLM
LLM Routing	LiteLLM	Single interface for 100+ models; automatic failover; cost tracking	Low - drop-in replacement
Short-term Memory	Redis 7+ (Redis Stack)	Sub-ms latency, TTL, pub/sub for streaming, JSON module	Low - structured data
Long-term Memory	Qdrant + PostgreSQL 16	Qdrant: production vector DB with hybrid search; PG: structured data	Low - standard SQL
Task Queue	Celery + Redis broker	Mature, well-documented, async task execution for HITL + background jobs	Low
Tool Sandbox	gVisor (runsc)	OS-level isolation for code exec; lower overhead than full VMs	Medium - vendor-specific
Observability	OpenTelemetry + Grafana + Vector	Vendor-neutral traces/metrics; LangSmith for LLM-specific debugging	Low - OTel is standard
Containerisation	Docker + Kubernetes 1.30+	Industry standard; Helm charts for reproducible deploys	Low
CI/CD	GitHub Actions + ArgoCD	GitOps workflow; ArgoCD for K8s sync	Low

3

PHASE 1 – FOUNDATION & LOCAL DEV SETUP

PHASE 1 · Foundation — Estimated: 3–4 days

Goal

A working local environment with all services running via Docker Compose, a skeleton FastAPI app, and a single end-to-end smoke test passing.

Repository Structure

```
agent-orchestration/
├── api/                # FastAPI app
│   ├── main.py
│   ├── routers/
│   └── schemas/
├── orchestration/     # LangGraph graphs & nodes
│   ├── graph.py
│   ├── nodes/
│   └── agents/
├── memory/            # Redis + Qdrant + PG clients
├── tools/              # Tool registry + implementations
├── hitl/              # Human-in-the-loop queue + UI
├── observability/     # OTEL config + dashboards
├── evals/             # Golden task evaluation suite
├── infra/             # Helm charts + K8s manifests
├── docker-compose.yml
├── docker-compose.prod.yml
└── Makefile
```

Step-by-Step: Phase 1

1 Create repo & virtualenv

Python 3.12+. Use uv for fast dependency management.

```
uv init agent-orchestration && cd agent-orchestration && uv venv
```

2 Install core dependencies

FastAPI, LangGraph, LiteLLM, Redis client, asyncpg, qdrant-client, celery.

```
uv add fastapi langgraph litellm redis asyncpg qdrant-client celery[redis]
```

3 Write docker-compose.yml

Services: postgres:16, redis:7, qdrant/qdrant:latest, grafana/grafana. Add health checks for all services. Mount volumes for persistence.

```
docker compose up -d && docker compose ps
```

- | | |
|---|---|
| 4 | Scaffold FastAPI app
main.py with POST /tasks and GET /tasks/{id}. Use Pydantic v2 models for TaskRequest and TaskResponse. Add /health endpoint.
uvicorn api.main:app --reload |
| 5 | Database migrations
Use Alembic for PostgreSQL schema. Tables: tasks, hitl_queue, memory_episodes, tool_calls.
alembic init alembic && alembic revision --autogenerate -m 'init' |
| 6 | Redis connection & test
Create memory/redis_client.py with async Redis client. Test PING and basic HSET/HGET.
python -c 'from memory.redis_client import get_redis; ...' |
| 7 | Qdrant collection setup
Create collection: agent_memories with vector size 1536 (OpenAI) or 1024 (Cohere). Enable payload indexing for tenant_id, task_id.
python memory/qdrant_setup.py |
| 8 | Smoke test
POST /tasks with a simple prompt. Verify 200 response with task_id. Check DB row created.
pytest tests/smoke/test_api.py -v |

AI Prompt for Phase 1 (use with Claude or Cursor)

"Generate a FastAPI app skeleton with: async POST /tasks endpoint accepting {prompt: str, tenant_id: str, config: dict}, returns {task_id: uuid, status: pending}. Use Pydantic v2. Add middleware for request logging with OpenTelemetry trace injection. Include /health and /metrics endpoints. Async PostgreSQL with asyncpg connection pool."

4

PHASE 2 – CORE ORCHESTRATION ENGINE

PHASE 2 · Orchestration Engine — Estimated: 8–10 days

LangGraph State Machine Design

The graph has five node types connected by conditional edges. The key design principle: **each node is a pure function** (state in → state out) for deterministic replay.

State Schema (AgentState)

Field	Type	Description
task_id	str	Unique task identifier
tenant_id	str	Multi-tenancy isolation key
original_request	str	Raw user prompt
plan	List[SubTask]	Planner output: ordered subtasks with dependencies
subtask_results	Dict[str, Any]	Results keyed by subtask_id
current_subtask	Optional[SubTask]	Currently executing subtask
tool_calls	List[ToolCall]	Full log of every tool invocation
reviewer_scores	Dict[str, float]	Reviewer confidence per subtask
hitl_required	bool	Flag set by reviewer when confidence < threshold
hitl_decision	Optional[str]	Human decision: approve / modify / reject
final_answer	Optional[str]	Synthesised output
error	Optional[str]	Last error for retry logic
metadata	Dict	Cost, latency, model used per step

Node Implementation Guide

Node	Responsibility	Implementation Note
supervisor_node	Entry point. Calls LLM to understand intent. Fetches top-3 similar past tasks from long-term memory. Enriches state with context. Routes to planner.	Model: Claude Sonnet. Prompt: system=SUPERVISOR_PROMPT, user=original_request+past_context
planner_node	Decomposes request into SubTask list with dependency DAG. Each SubTask has: id, agent_type, description, dependencies, estimated_tokens. Validates plan is acyclic before proceeding.	Force JSON output. Validate with Pydantic. Retry up to 3x on parse failure.

executor_node	Reads plan from state. Identifies ready subtasks (all deps completed). Dispatches parallel subtasks using asyncio.gather(). Updates subtask_results as each completes.	Use semaphore to limit concurrent LLM calls. Store partial results to Redis as checkpoints.
specialist_agent	Each specialist (Research/Data/Code/Writing) has its own system prompt + toolset. Runs ReAct loop: Think → Act (tool call) → Observe → repeat until done.	LiteLLM routes to cheapest model that can handle the task. Max 5 tool calls per subtask.
reviewer_node	Evaluates each specialist output against criteria: correctness, completeness, safety. Returns score 0–1. Sets hitl_required=True if score < 0.7 or sensitive_action detected.	Structured output: {score: float, issues: list, recommendation: str}

Conditional Edge Logic

Transition Point	Edge Logic
After supervisor_node	Always → planner_node
After planner_node	Plan valid → executor_node Plan invalid → supervisor_node (retry)
After executor_node	All subtasks done → reviewer_node Subtasks remaining → executor_node (loop)
After reviewer_node	All scores ≥ 0.7 → supervisor_node (synthesise) Low score → hitl_node
After hitl_node	Approved → executor_node (re-run subtask) Rejected → END with error
After supervisor (synthesise)	Always → END (stream final_answer)

Streaming Implementation

Use FastAPI's EventSourceResponse (sse-starlette). The LangGraph graph emits events at each node transition. A background task runs the graph; a Redis pub/sub channel bridges graph events to the SSE stream.

Event Type	Payload	When Emitted
task_accepted	{task_id, estimated_steps}	On POST /tasks
plan_created	{subtasks: [{id, type, description}]}	After planner_node
subtask_started	{subtask_id, agent_type}	Executor dispatches specialist
tool_call	{subtask_id, tool_name, input_summary}	Each tool invocation
subtask_completed	{subtask_id, duration_ms, cost_usd}	Specialist done
human_escalation	{subtask_id, reason, review_url}	Reviewer triggers HITL
human_decision	{subtask_id, decision}	Human acts in review UI
final_answer	{answer, total_cost_usd, total_duration_ms}	Synthesis complete
error	{message, retry_count}	Any unrecoverable failure

5

PHASE 3 – MEMORY SYSTEMS

PHASE 3 · Memory Layer — Estimated: 5–6 days

Short-Term Memory: Redis

1 Task context storage

Store full state snapshot after each node. Key: task:{id}:state. TTL: 24h.

```
HSET task:{id}:state field value EX 86400
```

2 Plan state tracking

Store plan JSON + completion status. Update atomically with Lua script.

```
HSET task:{id}:plan subtasks '{json}' completed '[]'
```

3 Intermediate results

Each subtask result: task:{id}:result:{subtask_id}. TTL: 1h after task completion.

4 SSE pub/sub channel

Publisher: graph nodes PUBLISH to task:{id}:events. Consumer: SSE endpoint SUBSCRIBE.

```
PUBLISH task:{id}:events '{event_json}'
```

5 Tool call cache

Cache tool outputs: tool:{tool_name}:{input_hash}. TTL: 5min (configurable per tool).

```
SET tool:web_search:{hash} '{result}' EX 300 NX
```

Long-Term Memory: Qdrant + PostgreSQL

Long-term memory stores **episodic traces** (full task histories), **semantic memories** (embedded summaries), and **user preferences**. Every memory is scoped by tenant_id.

1 Embedding strategy

Embed: task summary + plan structure + outcome quality. Use text-embedding-3-small (1536d) or Cohere embed-v3 (1024d). Batch embed for cost efficiency.

2 Qdrant collection schema

Payload fields: task_id, tenant_id, timestamp, quality_score, task_type, tool_list. Enable HNSW index. Enable payload index on tenant_id for filtered search.

```
qdrant_client.create_collection('agent_memories', vectors_config=VectorParams(size=1536, distance=Distance.COSINE))
```

3 Importance scoring

Score = 0.4 * outcome_quality + 0.3 * reuse_count + 0.3 * recency_decay. Run nightly job to prune memories with score < 0.2. Keeps collection lean.

4 Episodic memory retrieval

On new task: embed prompt → search Qdrant (top 3, filter by tenant_id, score > 0.75). Return full plan from matched episodes. Supervisor uses as few-shot examples.

```
qdrant_client.search('agent_memories', query_vector=embedding, limit=3,  
query_filter=Filter(must=[FieldCondition(key='tenant_id',  
match=MatchValue(value=tenant_id))]))
```

5 Memory consolidation job

Celery beat task runs every 6h. Clusters similar memories, merges into a single higher-quality entry. Prevents duplicate memory accumulation.

6 User preference memory

Separate Qdrant collection: user_preferences. Stores: preferred output format, domain knowledge, past feedback. Retrieved and injected into supervisor system prompt.

6

PHASE 4 – TOOL REGISTRY & SECURITY

PHASE 4 · Tool Registry & Sandboxing — Estimated: 4–5 days

Tool Registry Design

The registry is the single control plane for all tool interactions. It enforces: schema validation, rate limiting, cost tracking, audit logging, and sandbox enforcement for dangerous tools.

Tool	Category	Sandbox Required	Rate Limit	Cache TTL
web_search	Research	No	60/min/tenant	5 min
web_scrape	Research	No	30/min/tenant	10 min
python_execute	Code	YES – gVisor	10/min/tenant	None
file_read	Data	YES – read-only FS	100/min/tenant	1 min
file_write	Data	YES + HITL approval	20/min/tenant	None
sql_query	Data	YES – read-only role	50/min/tenant	2 min
api_call	Integration	No (allowlist only)	30/min/tenant	5 min
chart_generate	Visualisation	No	20/min/tenant	None
send_email	Action	HITL approval required	5/min/tenant	None

Sandboxing with gVisor

1 Install gVisor

Install runsc on host or K8s nodes. Configure Docker daemon to use runsc runtime.
`sudo apt-get install -y runsc && sudo runsc install`

2 Python execution sandbox

Each python_execute call spawns a Docker container with: --runtime=runsc, --read-only, --network=none, --memory=256m, --cpus=0.5, timeout=30s. Output captured from stdout/stderr only.
`docker run --runtime=runsc --read-only --network=none -m 256m --cpus=0.5 python:3.12-slim`

3 Allowlist for network tools

web_search and api_call: no sandbox, but domain allowlist enforced at DNS level via CoreDNS policy.

4 Tool result validation

All tool outputs pass through schema validator before returning to agent. Reject oversized outputs (> 50KB).

MCP Schema Integration

Register all tools following the **Model Context Protocol (MCP)** schema for future-proofing. This means any future MCP-compatible orchestrator can use your tools without changes.

```
# tools/registry.py snippet
class ToolDefinition(BaseModel):
    name: str
    description: str
    input_schema: dict # JSON Schema
    requires_sandbox: bool = False
    requires_hitl: bool = False
    rate_limit: RateLimit
    cache_ttl_seconds: Optional[int] = None
```

7

PHASE 5 – HUMAN-IN-THE-LOOP (HITL)

PHASE 5 · HITL & Review Workflow — Estimated: 4–5 days

HITL Architecture

- 1 Escalation trigger**
Reviewer node sets `hitl_required=True` when: score < 0.7, action is irreversible (file write, email send, API with side effects), or new unseen tool needed.
`state['hitl_required'] = True; state['hitl_reason'] = reason`
- 2 Create HITL queue entry**
PostgreSQL row: `id, task_id, subtask_id, tenant_id, reason, context_json, status=pending, sla_deadline=NOW()+10min, created_at`.
`INSERT INTO hitl_queue (task_id, subtask_id, ...) RETURNING id`
- 3 Celery task for SLA monitoring**
Beat schedule checks every 2 min for overdue reviews (`sla_deadline < NOW()`). Auto-decision: safe fallback (skip action) with `hitl_auto_fallback` flag.
- 4 Review UI**
Simple React app (or Next.js). Shows: original request, subtask details, specialist output, reviewer score + issues. Buttons: Approve / Edit & Approve / Reject.
`GET /hitl/queue → list pending reviews`
- 5 Human decision propagation**
`PATCH /hitl/{id} {decision: approve|reject, modified_output: ...}`. Celery worker picks up change, updates graph state in Redis, resumes LangGraph execution.
`graph.update_state(config, {'hitl_decision': decision})`
- 6 SLA auto-escalation**
If SLA breached: `status=auto_resolved, decision=safe_fallback`. Notify tenant via webhook. Log for audit. Continue graph with safe default.

8

PHASE 6 – OBSERVABILITY & TRACING

PHASE 6 · Observability — Estimated: 3–4 days

Three Pillars: Traces, Metrics, Logs

Signal	Tool	What to Capture	Where to View
Traces	OpenTelemetry → Grafana Tempo	Every node execution, tool call, LLM call. Include: tenant_id	Grafana Tempo Explorer, latency
Metrics	Prometheus → Grafana	task_count, task_duration_p99, tool_error_rate, llm_cost_per_token, hit_rate, sla_breach_rate	Grafana Prometheus
Logs	structlog → Loki	Structured JSON logs per node. Include trace_id for correlation	Grafana Loki
LLM Debug	LangSmith (dev only)	Full prompt/response for every LLM call. Latency breakdown. Token counts.	LangSmith UI
Cost Tracking	Custom Prometheus counters	\$/CPU-task, per model, per tenant. Daily budget alerts	Costs dashboard
Replay	Custom Python (replay.py)	Serialised state snapshots at each node. Can re-run from any checkpoint	CLI from python replay.py task-id X --from-node plan

Key Grafana Dashboards to Build

Dashboard	Key Metrics
Agent Overview	task_count by status, avg duration, success rate, cost today
Cost Dashboard	Cost by tenant, by model, by tool. Daily/weekly trend. Budget burn rate.
HITL Dashboard	Escalation rate, avg review time, SLA breach %, auto-resolved %.
Tool Performance	Tool call count, error rate, cache hit rate, p99 latency per tool.
Memory Health	Qdrant collection size, search latency, memory pruning rate.

9

PHASE 7 – PRODUCTIONISATION & KUBERNETES

PHASE 7 · Production Deployment — Estimated: 5–6 days

Kubernetes Architecture

K8s Resource	Config	Resource Request
api-deployment	FastAPI app. 3 replicas min. HPA on CPU+RPS. Liveness + readiness probes.	2 vCPU / 2GB × 3
worker-deployment	Celery workers for HITL + background jobs. 2 replicas. Scale on queue depth.	1 vCPU / 1GB × 2
redis-statefulset	Redis with Redis Sentinel for HA. PVC for persistence.	1 vCPU / 2GB × 3
qdrant-statefulset	Qdrant single node (dev) or cluster (prod). PVC.	2 vCPU / 4GB
postgres-statefulset	PostgreSQL with WAL streaming replica. PVC. Nightly backups to S3.	2 vCPU / 4GB × 2
grafana-deployment	Grafana + Prometheus + Loki + Tempo stack via kube-prometheus-stack Helm chart.	1 vCPU / 1GB

CI/CD Pipeline (GitHub Actions + ArgoCD)

- On PR: lint + test**
Run ruff, mypy, pytest. Coverage threshold: 80%. Block merge if failing.
`make lint test`
- On merge to main: build + push**
Build Docker image, tag with git SHA. Push to GCR/ECR. Scan with Trivy for CVEs.
`docker build -t gcr.io/project/agent-api:$SHA .`
- ArgoCD sync**
Update Helm values.yaml with new image SHA. ArgoCD auto-syncs to staging namespace. Manual sync required for production.
- Smoke test on staging**
Run golden task evaluation suite against staging. If pass rate < 90%, block prod deploy.
`python evals/run_golden_tasks.py --env staging`
- Production rollout**
ArgoCD progressive delivery: 10% → 50% → 100% traffic. Automated rollback on error rate spike.
- Secrets management**
All API keys in Kubernetes Secrets, sourced from GCP Secret Manager / AWS Secrets Manager. Never in code or env files.
`kubectl create secret generic llm-keys --from-literal=ANTHROPIC_API_KEY=...`

10

PHASE 8 – EVALUATION & TESTING

PHASE 8 · Evaluation Suite — Estimated: 3–4 days

Golden Task Evaluation Framework

The eval suite is your safety net. Run it before every deploy. Each golden task has: input prompt, expected plan structure, expected tool call sequence, and quality criteria for final answer.

Metric	Description	Target
Plan Accuracy	Does the generated plan match expected subtask types and dependencies?	≥ 95%
Tool Selection	Are the correct tools called for each subtask?	> 90%
Final Answer Quality	LLM-as-judge score vs reference answer (0–1 scale)	> 0.80
Cost per Task	Total USD across all LLM calls and tool calls	< \$0.10
Latency P95	End-to-end task completion time	< 30s
HITL Escalation Rate	% of tasks requiring human review on golden set	< 5%
Replay Fidelity	Re-running from checkpoint gives same final answer	100%

Test Categories

Test Type	Scope	Tools
Unit tests	Each node function, tool registry, memory client. Mock all external calls.	pytest + unittest.mock
Integration tests	Full graph runs with real Redis + Qdrant (Docker). No LLM calls (use fixtures).	docker-compose.test.yml
Contract tests	API schema validation. Verify SSE event format. Tool MCP schema parity.	openapi-schema-validator
Golden task evals	10–20 representative tasks with human-verified expected outputs.	custom eval runner
Load tests	50 concurrent tasks. Verify no state bleed between tenants.	locust
Adversarial tests	Prompt injection attempts, oversized inputs, tool call loops.	custom fixtures

11

TRADE-OFFS & DECISION MATRIX

Decision	Chosen Approach	Pros	Cons	When to Reconsider
Orchestration framework	LangGraph	State machine, cycles, checkpoints, human-in-the-loop	Learning curve, opinionated, not for simple tasks	Simple chains → use DSPy or custom
LLM routing	LiteLLM	Single interface, 100+ models, auto fallback	Static config, ~500ms latency, no fine-tuning	Highly dynamic → build custom router or use vLLM
Vector DB	Qdrant	Production-grade, hybrid search, filtering	More complex than Pinecone, needs Redis	Simple use case (≤50K vectors) → ChromaDB
Sandboxing	gVisor	OS-level isolation, K8s native	Higher overhead than VMs, not all syscalls supported	Need full OS support → use AWS Lambda or Firecracker
Memory design	Separate short + long term	Clear separation of concerns, optimized for each	Two systems to maintain, more complexity	Simple use case → single PostgreSQL with TTL
HITL queue	Celery + PostgreSQL	Mature, auditable, SLA tracking	Operational overhead; Celery config is complex	Highly dynamic → migrate HITL there
Reviewer agent	LLM-as-judge	No training needed, generalist model	Expensive (extra LLM call), slower	High volume → train a small classifier on y
Multi-tenancy	tenant_id in all objects	Simple, works with any DB	Must remember to filter everywhere	Very high scale → separate K8s namespaces

12

ALTERNATIVES COMPARISON

Orchestration Alternatives

Framework	Best For	Maturity	Learning Curve	Anthropic/OpenAI Native
LangGraph (chosen)	Complex stateful workflows, cycles, prototyping	High	High	Yes (Anthropic)
CrewAI	Quick multi-agent prototypes, role-based agents	Medium	Low	Partial
AutoGen	Conversational multi-agent, Microsoft ecosystem	High	Medium	OpenAI-native
DSPy	Optimisation-driven, self-improving prompts	Medium	High	No
Temporal	Mission-critical workflows, long-running, very high	High	High	No
Hand-rolled (asyncio)	Max control, zero deps, simple linear tasks	Low	Medium	Yes

Vector Database Alternatives

DB	Scale	Hybrid Search	Managed Cloud	Best For
Qdrant (chosen)	10M+ vectors	Yes (BM25 + dense)	Qdrant Cloud	Production, filtering
ChromaDB	< 1M vectors	Partial	No	Dev, prototypes
Pinecone	Unlimited	Yes	Fully managed	No-infra teams
pgvector (PostgreSQL)	Medium	Partial	RDS, Supabase	Single-DB teams
Weaviate	10M+ vectors	Yes (BM25 built-in)	WCS	Built-in summarisation
Milvus	Billions	Yes	Zilliz Cloud	Extreme scale

Observability Alternatives

Tool	LLM-Specific	Cost Tracking	Best Phase	Pricing
OTel + Grafana (chosen)	No	Custom	Production	Free (self-hosted)
LangSmith	Yes	Yes	Development	Free tier / paid
Braintrust	Yes	Yes	Evals	Pay per eval
Arize	Yes	Yes	ML Monitoring	Enterprise
Helicone	Yes	Yes, detailed	Cost focus	Usage-based
Weights & Biases	Partial	No	Experiments	Free / Enterprise

13

AI-ASSISTED DEVELOPMENT WORKFLOW

Since you are building this with AI, here is the exact workflow to move fast while producing production-quality code.

Recommended Tools by Task Type

Task	Best AI Tool	Prompt Strategy
Boilerplate code (FastAPI, Pydantic models)	Claude (claude.ai or API)	Provide schema + requirements in one shot. Ask for full file, not snippets.
LangGraph graph code	Claude	Paste your AgentState schema + node requirements. Ask for complete graph.
Infrastructure (docker-compose, Kubernetes manifests)	Claude	Describe each service + resource requirements. Ask for prod-ready config with secrets.
Debugging complex agent behaviour	Claude with full trace	Paste the OpenTelemetry trace JSON. Ask 'Why did the agent take this path?'
Writing tests	Claude or GitHub Copilot	Paste the function under test. Ask for unit + edge case tests.
Code review / security audit	Claude	Paste code + ask specifically about prompt injection, secret leakage, sandbox escape.
Architecture decisions	Claude	Describe your constraints. Ask for pros/cons table + recommendation.
Writing eval golden tasks	Claude	Provide 3 sample tasks. Ask to generate 15 diverse variants covering edge cases.

Minimal-Change Design Principles

To ensure future modifications require minimal changes, follow these principles throughout:

1	Dependency injection everywhere Pass Redis, Qdrant, and DB clients as parameters, never import globals. Swap any backend in tests by injecting a mock.
2	Provider-agnostic LLM calls All LLM calls go through LiteLLM. Never call anthropic.Anthropic() or openai.OpenAI() directly. Change model by updating MODEL_CONFIG dict.
3	Config-driven tool registry Tool definitions live in tools/config.yaml, not hardcoded Python. Add a new tool by adding a YAML entry + one Python function.
4	Event-sourced state The full task history is a sequence of events in Redis. Replay any task by replaying events. Never mutate state in place.
5	Feature flags Wrap new features in feature flags (LaunchDarkly or simple DB table). Enable per tenant without deploy.

PORTFOLIO PRESENTATION STRATEGY

What FAANG AI Teams Actually Evaluate

Criterion	What They Look For	How This Project Demonstrates It
Systems thinking	Can you design for scale, failure, observability?	Multi-layer architecture, HA Redis/PG, OTel tracing, replay system
LLM engineering	Do you know prompt design, evaluation, structured outputs?	Structured outputs, LLM-as-judge, golden task evals, model routing
Security awareness	Do you think about trust boundaries?	Tool sandboxing (gVisor), HITL for dangerous actions, tenant isolation
Production mindset	Have you shipped real systems?	K8s, CI/CD, health checks, SLA monitoring, graceful degradation
Agent fundamentals	Do you understand memory, planning, tool use?	Substate, long-term memory, episodic retrieval, ReAct loop, importance scoring
Code quality	Is the code maintainable?	DI everywhere, config-driven tools, 80% test coverage, type hints

Priority Build Order for Interviews

1	Must-have (MVP, 4 weeks) Phase 1–4: working API + LangGraph graph + specialist agents + tool registry + Redis memory. Golden task eval suite. Basic Grafana dashboard. README with architecture diagram.
2	High impact (week 5) SSE streaming + HITL queue + review UI. This is the most demo-able feature. Shows end-to-end human-AI collaboration.
3	Strong differentiator (week 6) Long-term memory with episodic retrieval. Show the system gets smarter with each task. This impresses most interviewers.
4	Elite level (week 7–8) Tool sandboxing (gVisor) + multi-tenancy + K8s deploy. Cost dashboard showing per-task USD. Replay system demo.

Interview Talking Points

- "I chose LangGraph over CrewAI because I needed deterministic replay and the ability to resume from any checkpoint in production."
- "The reviewer node uses LLM-as-judge, but I benchmarked it — it costs \$0.003 per task and catches 94% of low-quality outputs."
- "All code execution runs in gVisor sandboxes. Even if the LLM generates malicious code, the worst case is a container restart."
- "Long-term memory uses importance scoring. Memories below 0.2 are pruned nightly, which keeps Qdrant lean and search fast."
- "The SSE streaming endpoint was the hardest part — bridging LangGraph's async state machine to HTTP streaming required a Redis pub/sub intermediary."

Good luck with the applications. Build the MVP first — ship something working, then layer on complexity. The architectural understanding you gain building this will carry through every interview, regardless of which specific tools the company uses.